

ESTRUCTURAS DE DATOS

Ayudantía 2: Análisis de Algoritmos y TDAs

Ignacia Nettle Oliveri
inettle@usm.cl

BLOQUE 1

Notación Big-O

Cota Superior y Técnicas de Análisis

Notación Big-O: Cota Superior

Big-O Notation

La notación Big-O (O "grande") es usada para describir la **complejidad temporal** de un algoritmo.

- ▶ Describe el **orden de crecimiento** en términos del tamaño de la entrada, no su valor exacto.
- ▶ Entrega la **cota superior** que toma un algoritmo según el tamaño de la entrada, es por esto que consideraremos el **peor caso** posible para encontrar su complejidad en notación Big-O.

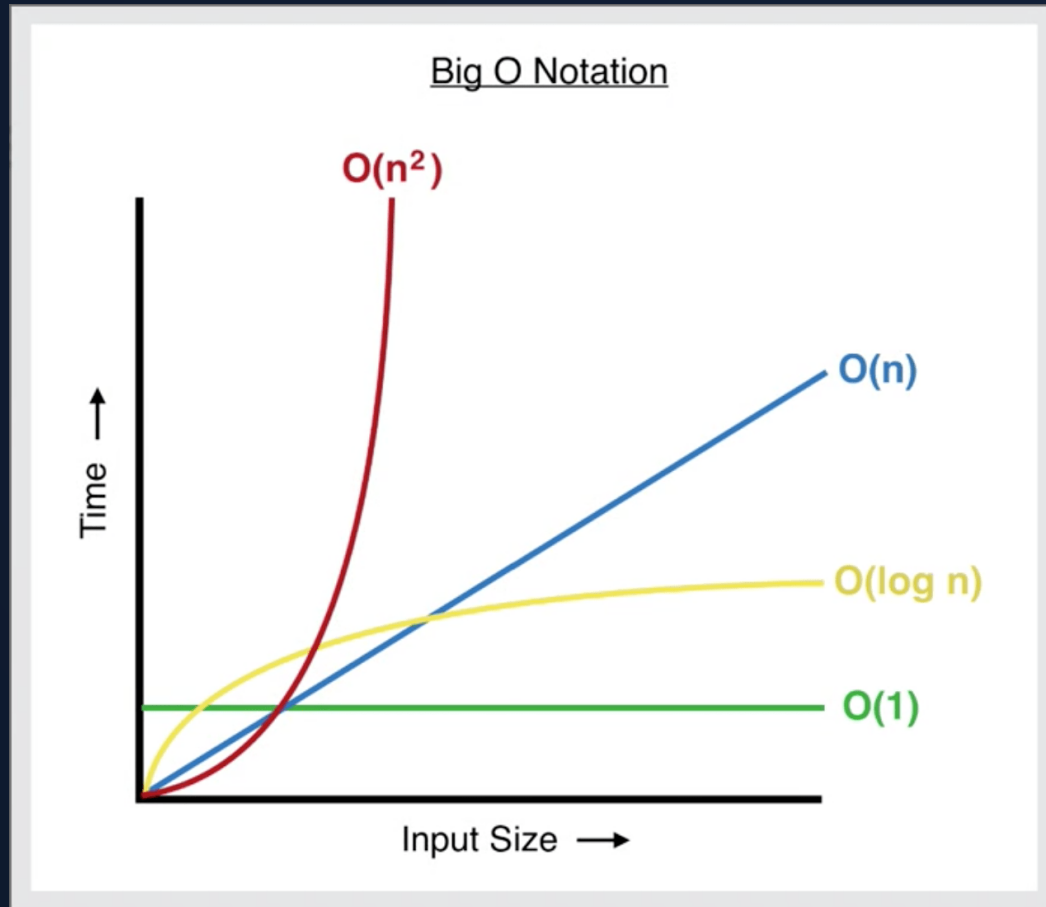
Big-O Definition

Dadas dos funciones $f(n)$ y $g(n)$, diremos que $f(n)$ es de orden $O(g(n))$ si existen las constantes $c > 0$ y $n_0 \geq 0$ tal que $f(n) \leq c \cdot g(n)$, $\forall n \geq n_0$.

En otras palabras, $f(n)$ es $O(g(n))$ si $f(n)$ **crece a lo más tan rápido como** $c \cdot g(n)$ **para todo** $n \geq n_0$.

Notar que nos interesa la función $g(n)$ **más pequeña** tal que $f \in O(g(n))$.

Notación Big-O: Cota Superior



Propiedades Importantes

- ▷ **Regla de la Suma:**
 $f_1 \in O(g), f_2 \in O(h) \Rightarrow f_1 + f_2 \in O(\max(g, h))$
- ▷ **Regla del Producto:**
 $f_1 \in O(g), f_2 \in O(h) \Rightarrow f_1 \cdot f_2 \in O(g \cdot h)$
- ▷ **Regla del Límite:** $f_1 \in O(g), f_2 \in O(h)$, si existe el límite

$$\lim_{n \rightarrow \infty} \frac{g}{h} = k$$

Si $k \neq 0$ y $k \neq \infty \Rightarrow O(g) = O(h)$

Si $k = 0 \Rightarrow g \in O(h), h \notin O(g)$.

Notación Big-O: Técnicas de Análisis

Operaciones Básicas y Condicionales

Diremos que las operaciones básicas (asignación, suma, resta, comparación, etc) siempre serán de orden $O(1)$, es decir, son **constantes**.

La complejidad de los `if-else` dependen de la expresión en la condición. Si bien por lo general estas serán de orden $O(1)$ ya que suelen ser comparaciones, es posible que existan casos en los que evaluar la condición tenga comportamiento lineal o logarítmico, por lo que hay que tener precaución al analizar estos casos.

Ejemplo

```
string s = "hola mundo";
string substr = "mundo";
if(s.find(substr) != string::npos){
    cout << "Substring found." << endl;
} else {
    cout << "Not found." << endl;
}
```

En este caso, la complejidad de este código es $O(n \times m)$ siendo n el largo de `s` y m el largo del `substr`, ya que *en el peor caso*, la función `find()` revisa por cada caracter, si el substring puede empezar en esa posición.

Notación Big-O: Técnicas de Análisis

Bucles

Analizar la complejidad de los bucles es un poco más complejo puesto que tenemos que tener en cuenta tres puntos clave:

- ▶ La condición de salida
- ▶ El comportamiento del iterador
- ▶ El cuerpo del bucle

Los primeros dos puntos están fuertemente relacionados, puesto que la combinación de ambas nos dice **cuántas iteraciones se realizarán**.

La condición de salida se comportará como la ecuación que necesitamos despejar para encontrar la cantidad de iteraciones k , mientras que el comportamiento del iterador nos dirá la función, digamos $f(i)$, asociada a k .

```
for(int i = 0; i < n; i++){  
    /* code here */  
}
```

En este caso, el iterador se comporta **linealmente** $f(i) = i$: si estamos en la iteración k , el valor del iterador también será k , por lo que al reemplazar en la ecuación obtenemos $k < n$, es decir, el código es de orden $O(n)$.

Notación Big-O: Técnicas de Análisis

Complejidad Logarítmica

```
for(int i = 1; i < n; i *= 2){  
    /* code here */  
}
```

En este caso, el iterador se comporta exponencialmente $f(i) = 2^i$. Es decir, en la iteración k , el valor del iterador será 2^k .

Si reemplazamos la función en la ecuación y luego despejamos en función de k :

$$\begin{aligned} 2^k &< n \\ k &< \log_2 n \end{aligned}$$

Logaritmos de distinta base difieren entre sí por una **constante**¹, por lo que en notación Big-O podemos decir que logaritmos en cualquier base son de orden $O(\log n)$.

Complejidad Raíz Cuadrada

```
for(int i = 0; i*i < n; i++){  
    /* code here */  
}
```

Por otro lado, si bien en este caso el iterador se comporta linealmente $f(i) = i$, al reemplazar en la ecuación obtenemos:

$$k^2 < n$$

Despejando en función de k :

$$k < \sqrt{n}$$

Por lo que, la complejidad del código es de orden $O(\sqrt{n})$.

1: Recordar la propiedad de cambio de base de logaritmos.

Ejercicio

Dados dos algoritmos A y B, ¿cuál es mejor en términos de uso de recursos?

Algoritmo A

```
int sum = 0;
for(int i = 0; i < 2*n; i++){
    int c = i;
    while(c > 0){
        sum++;
        c = c/2;
    }
}
```

Algoritmo B

```
for(int j = 0; j <= n; j++){
    int i = 1;
    int sum = 0;
    while(sum <= n){
        sum = sum+i;
        i++;
    }
}
```


Ejercicio

Análisis Algoritmo A

```
int sum = 0;
for(int i = 0; i < 2*n; i++){
    int c = i;
    while(c > 0){
        sum++;
        c = c/2;
    }
}
```

El bucle externo crece linealmente, por lo que podemos representar su valor luego de k iteraciones mediante la función $f(k) = k$. Reemplazando en la condición de salida, obtenemos:

$$k < 2n$$

Recordar que en notación Big-O no nos importan las constantes, por lo que el bucle externo es de orden $O(n)$.

Ejercicio

Análisis Algoritmo A

```
int sum = 0;
for(int i = 0; i < 2*n; i++){
    int c = i;
    while(c > 0){
        sum++;
        c = c/2;
    }
}
```

En el bucle interno, tenemos un iterador **c** que parte desde el valor **i** y luego es dividido por 2 en cada iteración, por lo que podemos expresar el valor de *c* en cada iteración como: $f(k) = i \cdot 2^{-k}$

Ya que estamos trabajando con **división entera** podemos reformular la condición de salida de tal forma que la pregunta sea: *¿Cuántas veces podemos dividir *c* por 2 hasta que el resultado sea menor que 1?*¹

1: Esta reformulación se hace para evitar el caso de logaritmo de 0, que tiene comportamiento indefinido.

Ejercicio

Análisis Algoritmo A

```
int sum = 0;
for(int i = 0; i < 2*n; i++){
    int c = i;
    while(c > 0){
        sum++;
        c = c/2;
    }
}
```

Con esta condición de salida equivalente, podemos construir la siguiente ecuación:

$$i \cdot 2^{-k} \geq 1$$

$$i \geq 2^k$$

$$\log_2 i \geq k$$

Dado que en el peor caso $i = n$, el bucle interno es de orden $O(\log n)$.

Entonces, la complejidad del algoritmo completo es de $O(n \log n)$

Ejercicio

Análisis Algoritmo B

```
for(int j = 0; j <= n; j++){  
    int i = 1;  
    int sum = 0;  
    while(sum <= n){  
        sum = sum+i;  
        i++;  
    }  
}
```

Por otro lado, ya que el iterador `sum` del bucle interno aumenta en un valor distinto en cada iteración, no es inmediatamente obvio cómo expresar el valor de `sum` tras k iteraciones.

Luego de k iteraciones:

$$sum = 1 + 2 + 3 + \dots + (k - 2) + (k - 1) + k$$

Ejercicio

Análisis Algoritmo B

```
for(int j = 0; j <= n; j++){  
    int i = 1;  
    int sum = 0;  
    while(sum <= n){  
        sum = sum+i;  
        i++;  
    }  
}
```

Notar que estos números son **equiespaciados**: la diferencia entre dos números consecutivos siempre es 1.

Debido a esto, podemos ir emparejando los números de extremos opuestos:

$$(1, k), (2, k-1), (3, k-1) \dots$$

de tal forma que cada par **siempre suma** $k+1$.

Ya que tenemos k números, podemos formar como máximo $\frac{k}{2}$ pares, por lo tanto, la función para **sum** después de k iteraciones es:

$$f(k) = \frac{k(k+1)}{2}$$

Ejercicio

Análisis Algoritmo B

```
for(int j = 0; j <= n; j++){  
    int i = 1;  
    int sum = 0;  
    while(sum <= n){  
        sum = sum+i;  
        i++;  
    }  
}
```

Reemplazando en la condición de salida, obtenemos:

$$\frac{k(k+1)}{2} \leq n$$
$$\frac{k^2}{2} + \frac{k}{2} \leq n$$

Recordar que en notación Big-O sólo nos interesa el término de mayor orden, ignorando las constantes:

$$k^2 \leq 2$$
$$k \leq \sqrt{n}$$

Por lo que, el bucle interno tiene orden $O(\sqrt{n})$.
Dado esto, el orden del algoritmo completo es $O(n\sqrt{n})$.

Ejercicio

¿Cuál es mejor?

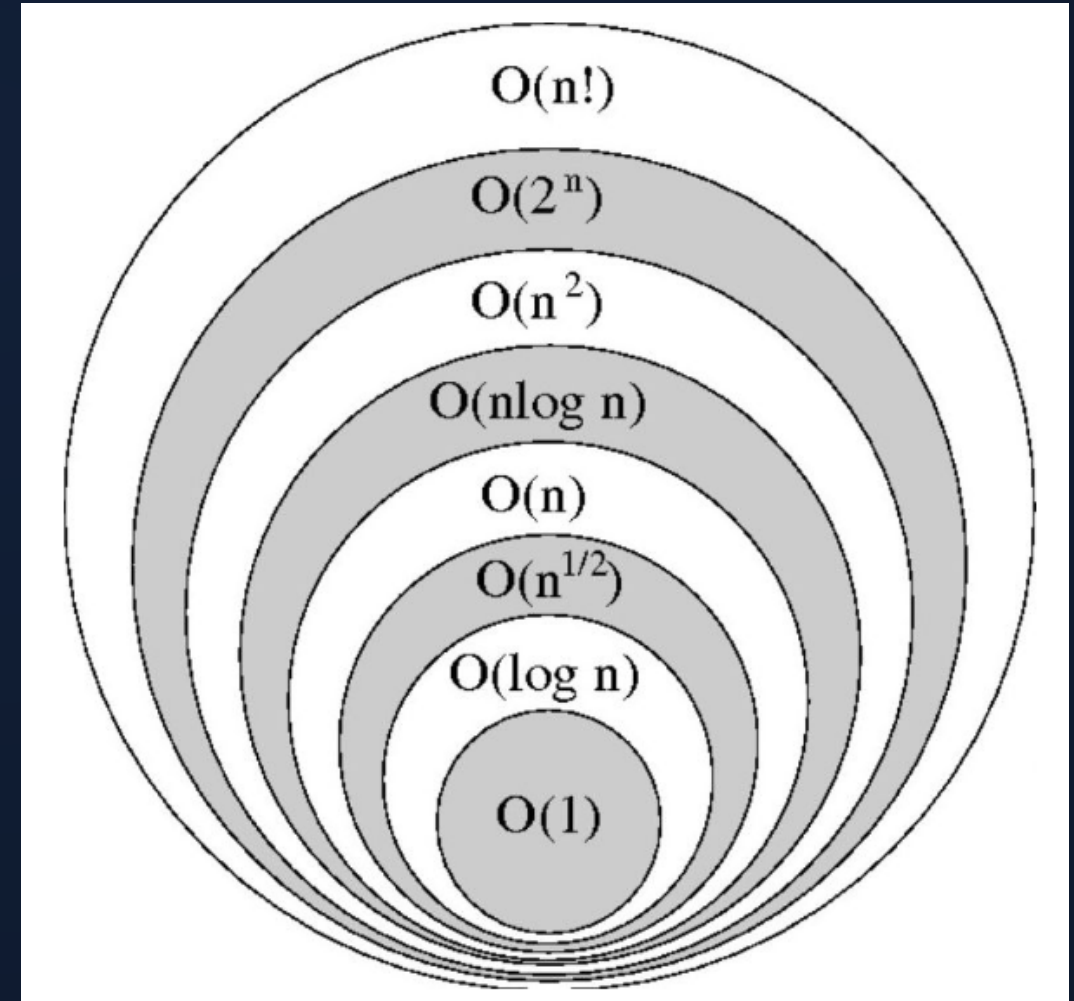
El Algoritmo A tiene complejidad $O(n \log n)$ mientras que el Algoritmo B tiene complejidad $O(n\sqrt{n})$.

Recordando la jerarquía de inclusión de funciones, podemos decir que el Algoritmo A es más eficiente y por lo tanto, la mejor opción.

Desafío: Usando la propiedad del límite, demostrar que $O(n \log n) \in O(n\sqrt{n})$:

$$\lim_{n \rightarrow \infty} \frac{n \log n}{n\sqrt{n}}$$

Hint: L'Hopital puede ser útil para resolver este límite.



BLOQUE 2

TDA_s

Abstractos, Lineales, Listas Basadas en Arreglos

Tipos de Datos Abstractos

Recordando...

Un **tipo** en un lenguaje de programación es una colección de valores que una variable de ese tipo puede tomar. Los tipos se separan en dos categorías:

- Tipos simples: `int`, `float`, `double`, `bool`
- Tipos compuestos: arreglos y structs

Un **tipo de dato** es un **tipo** junto con una **colección de operaciones** para manipularlo. Por ejemplo:

- `int` tiene todas las operaciones aritméticas.
- `bool` tiene todas las operaciones lógicas.

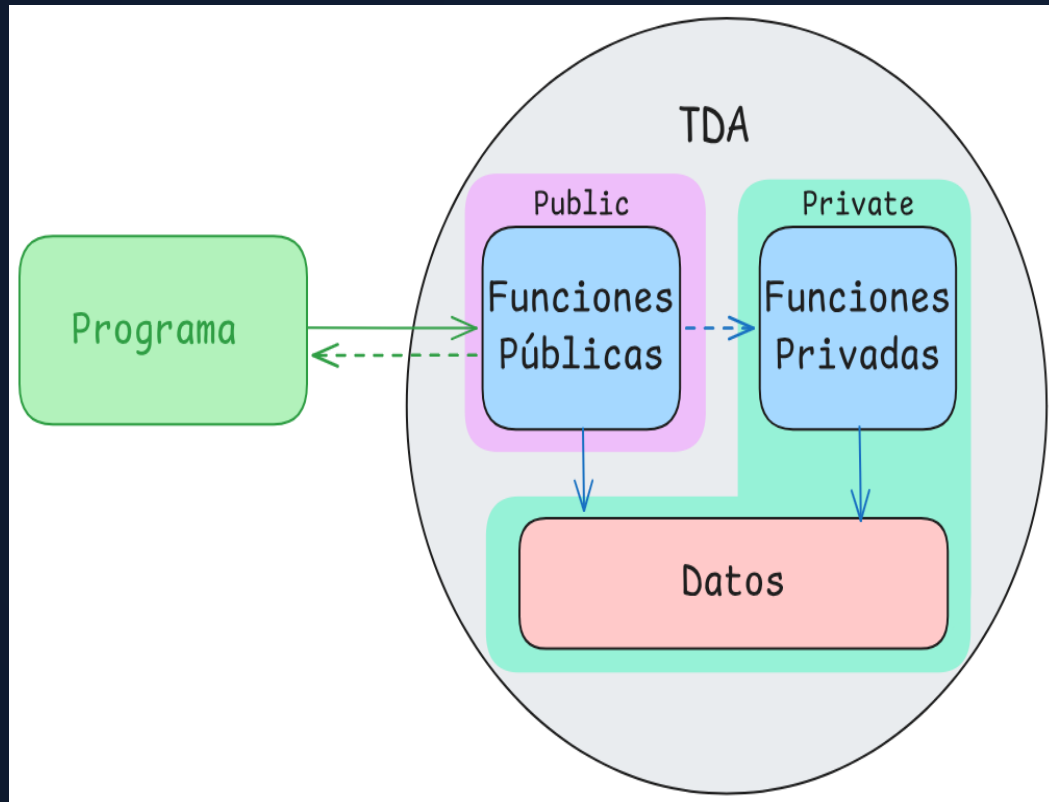
Definición

Un tipo de dato **abstracto** es, a grandes rasgos, un *modelo conceptual*. Se enfocan en **qué hace la estructura**, no el **cómo lo hace**.

Los TDAs permiten cambiar la implementación de operaciones **sin afectar cómo el resto del programa las usa**.

Esto es vital para producir código *mantenible* e implementar grandes sistemas de software.

Tipos de Datos Abstractos



Structs vs Clases

En `C++` se pueden usar tanto `Structs` como `Clases` para definir un TDA. La única diferencia entre ambas es:

- ▷ `Structs`: los datos están por defecto en el `public`, es decir, el usuario puede acceder y modificar los datos como desee.
- ▷ `Clases`: los datos están por defecto en el `private`, es decir, estos están ocultos para el usuario. La única forma de acceder a ellos es mediante los métodos del mismo.

Por convención, usaremos las **Clases** para definir un TDA. Los structs serán usados para agrupar variados datos en un solo "contenedor".

Ojo: Si tengo un puntero apuntando a un struct, para acceder a sus atributos desde el puntero se usa `->` en lugar de `.`

Tipos de Datos Abstractos: Ejemplo

```
class Playlist{
private:
    string nombre;
    int numCanciones;
    string canciones[10];
public:{
    Playlist(string n) {nombre = n; numCanciones = 0;}
    ~Playlist() {};
    void agregar(string c);
    void eliminar(string c);
    void mostrar();
}
}
```

Para **implementar** los métodos se usa la siguiente sintáxis:

```
tipoMetodo NombreClase::metodo(parametros){/*code*/}
```

Por ejemplo, implementaremos el método `agregar()`:

```
void Playlist::agregar(string c){
    if(numCanciones < 10){
        canciones[numCanciones] = c;
        numCanciones++;
        cout << n << "agregada a " << nombre << endl;
    } else {
        cout << "Playlist llena." << endl;
    }
}
```

Ejercicio

La tienda de videojuegos **USMPlay** ofrece una gran cantidad de títulos para las consolas **PlayStation, Xbox y Nintendo**.

Debido al crecimiento del catálogo, la tienda necesita automatizar la gestión de su inventario. Actualmente, los videojuegos se registran de forma manual, lo que dificulta mantener la información organizada y de rápido acceso.

Para solucionar el problema, se ha comenzado a desarrollar un sistema de inventario que utiliza la clase **Inventario**, que será la encargada de **mantener registro del stock disponible y organizar** los juegos de la tienda **según la consola** en la que se encuentran disponibles.

Ejercicio

La administración de USMPlay te ha pedido que implementes los métodos de `Inventario` de modo que cumplan con los objetivos descritos.

Considera que tienes a tu disposición el archivo `resumen.txt` que describe las compras y ventas realizadas por la tienda un día cualquiera: su primera línea es un entero `n` seguido de `n` líneas que describen los eventos de la forma:

```
Videojuego-Consola;Cantidad
```

Notar que un videojuego puede aparecer **varias veces** en el archivo y puede estar disponible en **más de una consola**. El listado de consolas con las que trabaja USMPlay son: **PlayStation**, **Xbox** y **Nintendo**.

`Cantidad` es positivo para compras y negativo para ventas. Por simplicidad, en caso de haber stock negativo simplemente setearlo a 0.

Ejercicio

Clase `Inventario`

```
class Inventario{
    private:
        Consola PlayStation, Xbox, Nintendo;
        void aumentarCapacidad(Videojuego *&v, int &cap);
    public:
        Inventario();
        ~Inventario();
        void agregar(string nom, string cons, int cant);
        void actualizar(string nom, string cons, int cant);
        void consultar(string nom, string cons);
        void stockConsola(string cons);
        void guardarInventario(string out_name);
}
```

Structs Adicionales y Consideraciones

```
Struct Videojuego{
    string nombre;
    int cantidad;
}

Struct Consola{
    int numVideojuegos;
    int capacidad = 10;
    Videojuego *videojuegos = new Videojuego[capacidad];
}
```

- ▷ Se pueden crear métodos auxiliares.
- ▷ `guardarInventario` : genera un archivo `.txt` de nombre `out_name` que contiene la información del inventario completo.

CIERRE

¡Eso es todo 🧐!

En este [repositorio](#) podrán encontrar las soluciones a los ejercicios.